

# Báo Cáo Triển Khai Chuyên Sâu: Nền Tảng Mô Phỏng Phân Bỏ Tác Vụ AI Dựa Trên Tối Ưu Hóa Năng Lượng Đa Mục Tiêu

## Tóm Tắt Định Hướng Triển Khai Thực Tiễn

Quá trình triển khai các tác vụ huấn luyện trí tuệ nhân tạo (AI Training Jobs) quy mô lớn tại các trung tâm dữ liệu (Data Center) hiện đại đang tạo ra một nghịch lý về tài nguyên: sự gia tăng theo cấp số nhân của nhu cầu tính toán dẫn đến chi phí năng lượng khổng lồ, trong khi hiện tượng phân mảnh tài nguyên (Resource Fragmentation) lại khiến hàng loạt GPU rơi vào trạng thái nhàn rỗi. Để giải quyết bài toán này, hệ thống cần một cơ chế lập lịch thông minh, tối ưu hóa đồng thời hai mục tiêu xung đột: giảm thiểu tổng năng lượng tiêu thụ và đảm bảo các cam kết về mức dịch vụ (SLA).

Báo cáo này tập trung vào bản thiết kế kiến trúc kỹ thuật và kế hoạch triển khai cụ thể cho Giai đoạn 1. Hệ thống mô phỏng sẽ được xây dựng trên nền tảng **CloudSim Plus** (phiên bản hiện đại và tối ưu hóa của CloudSim), kết hợp với thư viện **MO-Gymnasium** thông qua cầu nối **Py4J**. Thiết kế này cho phép đánh giá chính xác mức tiêu thụ năng lượng và hiệu suất của các thuật toán Học tăng cường đa mục tiêu (MORL) một cách hoàn toàn độc lập. Đặc biệt, kiến trúc Giai đoạn 1 được thiết kế tinh chỉnh để phục vụ trực tiếp cho Giai đoạn 2: tích hợp sẵn các bộ lập lịch cơ sở (baseline schedulers), công cụ chia tách kịch bản tải, và cổng theo dõi siêu tham số (hyperparameters), tạo tiền đề vững chắc cho quá trình thực nghiệm và đánh giá sau này.

## 1. So Sánh Kiến Trúc Mô Phỏng: KWOK vs. CloudSim

Để xác định công cụ phù hợp nhất cho Giai đoạn 1 (tập trung vào mô hình toán học và thuật toán tối ưu), dưới đây là bảng so sánh chi tiết giữa hai cách tiếp cận:

| Tiêu chí          | KWOK (Kubernetes WithOut Kubelet)                                        | CloudSim (đặc biệt là CloudSim Plus)                                                 | Lựa chọn cho Giai đoạn 1                                                                     |
|-------------------|--------------------------------------------------------------------------|--------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Bản chất hệ thống | Giả lập Node ở tầng API Kubernetes. Sử dụng bộ lập lịch (Kube-scheduler) | Trình mô phỏng sự kiện rời rạc (Discrete-event simulator) chuyên dụng cho nghiên cứu | <b>CloudSim</b> vượt trội vì cho phép kiểm soát hoàn toàn thời gian mô phỏng (không phải chờ |

|                          | thực tế.                                                                                                                        | Cloud.                                                                                                                                                  | realtime).                                                                                              |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Mô hình Năng lượng       | <b>Không có sẵn.</b> Cần cài đặt thêm các công cụ phức tạp như Kepler/Prometheus và phải xây dựng mô hình ngoại suy để giả lập. | <b>Tích hợp sẵn (Built-in).</b> Cung cấp sẵn các lớp PowerModel, PowerDatacenter, PowerHost tính toán chi tiết điện năng tiêu thụ dựa trên tải CPU/RAM. | <b>CloudSim</b> là lựa chọn an toàn và chính xác hơn cho việc trích xuất vector phần thưởng năng lượng. |
| Triển khai Thuật toán AI | Khá phức tạp. Phải viết HTTP Webhook (Extender) để can thiệp vào quá trình Bind của Kubernetes.                                 | Dễ dàng tùy chỉnh. Cung cấp các giao diện mở (Interfaces) để ghi đè (override) logic cấp phát tài nguyên VmAllocationPolicy.                            | <b>CloudSim</b> cung cấp môi trường Sandbox lý tưởng để các thuật toán RL tập trung học hỏi.            |
| Mở rộng sang Giai đoạn 2 | Sẵn sàng 100% vì hệ thống đã nằm trên Kubernetes.                                                                               | Cần phải chuyển đổi logic (Porting) từ mô hình CloudSim sang mã nguồn Kubernetes Extender.                                                              | Cần tốn thêm công sức ở Giai đoạn 2, nhưng tạo ra nền tảng lý thuyết vững chắc hơn ở Giai đoạn 1.       |

**Kết luận:** Việc sử dụng **CloudSim Plus** (được viết trên Java 17+) thay cho KWOK sẽ loại bỏ hoàn toàn gánh nặng phải thiết lập hạ tầng Kubernetes cục bộ (như etcd, API Server) và hệ thống Telemetry rườm rà. CloudSim Plus cho phép sinh viên đi thẳng vào lõi của bài toán: thiết kế thuật toán, tính toán năng lượng và thiết lập baseline so sánh.

## Kiến Trúc Hệ Thống Tổng Thể Mới

Khung triển khai được chia thành bốn phân hệ cốt lõi, liên kết luồng dữ liệu từ Java (Mô phỏng) sang Python (Trí tuệ nhân tạo):

1. **Phân hệ Mô phỏng Hạ tầng & Năng lượng (Java - CloudSim Plus):** Tạo lập Data

Center ảo, định nghĩa các máy chủ (Host) có gắn GPU, định nghĩa mô hình tiêu thụ điện (PowerModel) theo đặc tả phần cứng vật lý.

2. **Phân hệ Xử lý Workload (Java Trace Parser):** Trích xuất dữ liệu từ Alibaba GPU Cluster Trace, chuyển đổi thành các kịch bản tải (Low, High, Burst) dưới dạng Cloudlet và yêu cầu tài nguyên 컨테이너 (Vm/Container).
3. **Phân hệ Cầu nối Giao tiếp (Py4J Gateway):** Xây dựng cầu nối RPC (Remote Procedure Call) giữa JVM (Java) và Python. Python sẽ lấy trạng thái của Data Center và trả về quyết định lập lịch, đồng thời ghi nhận log cho quá trình tinh chỉnh siêu tham số.
4. **Phân hệ Tối ưu hóa MORL (Python - MO-Gymnasium):** Tác nhân AI tương tác với môi trường mô phỏng thông qua API chuẩn, tối ưu hóa hàm mục tiêu kép (Năng lượng và SLA).

---

## 2. Phân Hệ Mô Phỏng Hạ Tầng: Xây Dựng Data Center Với CloudSim Plus

Dự án sử dụng **CloudSim Plus**, một bản fork hiện đại của CloudSim, giải quyết các hạn chế về mã nguồn lặp lại, gia tăng tốc độ mô phỏng và hỗ trợ độ chính xác cao hơn cho các nghiên cứu cấp doanh nghiệp.

### 2.1. Cấu hình phần cứng và GPU (Host & Vm)

Trong CloudSim Plus, một cụm Kubernetes sẽ được ánh xạ thành một Datacenter. Các Worker Node được ánh xạ thành các đối tượng Host. Các tác vụ AI (Pods) được đóng gói dưới dạng Cloudlet chạy bên trong các Vm (hoặc Container). Việc lựa chọn kỹ thuật phân bổ (như Space-shared hoặc Time-shared) sẽ ảnh hưởng trực tiếp đến thời gian thực thi và năng lượng tiêu thụ, trong đó Space-shared thường cung cấp hiệu năng cô lập tốt hơn cho các tác vụ AI.

Để giả lập GPU cho AI Workload, chúng ta cấu hình các tài nguyên mở rộng. Một Host được khởi tạo với các thông số PE (Processing Elements - CPU Cores), RAM, BW (Băng thông) và Pe chuyên dụng cho GPU.

Java

```
// Khởi tạo một Host với mô hình tiêu thụ năng lượng
List<Pe> peList = new ArrayList<>();
// Thêm 96 CPU cores và 8 GPU cores ảo
for (int i = 0; i < 96; i++) peList.add(new PeSimple(1000)); // 1000 MIPS

Host host = new HostSimple(ram, bw, storage, peList);
// Gắn mô hình năng lượng (Ví dụ: Server tiêu thụ từ 150W (Idle) đến 350W (Max))
```

```
PowerModel powerModel = new PowerModelHostSimple(350, 150);  
host.setPowerModel(powerModel);
```

## 2.2. Đo Lường Năng Lượng Động (Dynamic Power Metrics)

Điểm mạnh tuyệt đối của CloudSim là khả năng theo dõi năng lượng mà không cần công cụ ngoại vi. Tổng điện năng tiêu thụ ( $E$ ) của một Node được tính toán tại mỗi chu kỳ mô phỏng dựa trên phần trăm sử dụng CPU ( $U$ ) theo công thức tuyến tính hoặc phi tuyến tính tích hợp sẵn:

$$P(U) = P_{idle} + U \times (P_{max} - P_{idle})$$

Và tổng năng lượng được tích phân theo thời gian thực thi  $t$ :

$$E = \int_{t_0}^{t_1} P(U(t))dt$$

Mức tiêu thụ này sẽ được trích xuất trực tiếp thông qua hàm `host.getPowerModel().getPower()` để chuyển về cho thuật toán RL làm phần thưởng.

## 2.3. Xây Dựng Baseline: Mô Phỏng Kubernetes Default Scheduler

Để phục vụ trực tiếp cho nhiệm vụ "Đánh giá & So sánh" ở Giai đoạn 2, ngay trong Giai đoạn 1, hệ thống phải cung cấp một cơ chế giả lập Kube-scheduler mặc định làm đường cơ sở (baseline). Kube-scheduler mặc định thường có thời gian lên lịch nhanh nhưng lại thiếu khả năng tối ưu hóa đa mục tiêu chuyên sâu về điện năng.

Trong CloudSim Plus, chúng ta sẽ mở rộng lớp `VmAllocationPolicy` để tạo ra `VmAllocationPolicyK8sDefault`. Policy này mô phỏng hai pha cốt lõi của Kube-scheduler:

1. **Filter (Sàng lọc):** Loại bỏ các Host không đủ CPU/RAM/GPU.
2. **Prioritize (Chấm điểm):** Áp dụng thuật toán `LeastRequestedPriority` (ưu tiên Node có tỷ lệ tài nguyên trống cao nhất) hoặc `BalancedResourceAllocation` (cân bằng mức sử dụng CPU/RAM).

Sự tồn tại của class này đảm bảo rằng ở Giai đoạn 2, chúng ta có thể chạy cùng một bộ workload qua hai hệ thống (K8s Default và DRL Agent) để thu thập dữ liệu đối chiếu chính xác.

## 3. Phân Hệ Dữ Liệu: Xử Lý Alibaba GPU Trace Cho Các Kịch Bản Tải

Để đảm bảo thuật toán RL được huấn luyện sát với thực tế và sẵn sàng cho việc kiểm thử tính bền vững ở Giai đoạn 2, bộ dữ liệu **Alibaba Cluster Trace v2023** không chỉ được đọc vào

nguyên bản mà phải được phân lớp thành các kịch bản môi trường (Environment Scenarios).

### 3.1. Phân Tách Dữ Liệu Thành Các Kịch Bản Huấn Luyện

Sinh viên sẽ xây dựng lớp AlibabaTraceReader kế thừa từ TraceReaderAbstract của CloudSim Plus, đồng thời tích hợp logic lọc dữ liệu dựa trên mật độ tác vụ theo thời gian (creation\_time).

- **Tải thấp (Low Load):** Trích xuất các khung giờ đêm, khoảng cách giữa các tác vụ thưa thớt. Tác nhân RL học cách tắt (Standby/Sleep) các Host để tiết kiệm năng lượng.
- **Tải cao (High Load):** Trích xuất các khung giờ làm việc cốt lõi, tỷ lệ sử dụng tài nguyên > 80%. Tác nhân RL học cách nén (Bin-packing) để tránh vi phạm SLA.
- **Tải đột biến (Burst Load):** Kịch bản tiêm các tác vụ có nhu cầu num\_gpu lớn chèn ngang, buộc tác nhân RL học cách đối phó với sự phân mảnh tài nguyên (Resource Fragmentation) đột ngột.

### 3.2. Ánh Xạ Thông Số Kỹ Thuật

Trường dữ liệu sẽ được ánh xạ chính xác như sau:

- cpu\_milli, memory\_mib  $\rightarrow$  Số lượng PE và RAM của đối tượng Vm.
- num\_gpu  $\rightarrow$  Yêu cầu tài nguyên đồ họa đặc thù của Vm.
- qos (Burstable, Latency Sensitive)  $\rightarrow$  Được truyền sang Python để làm hệ số  $\lambda$  trong hàm phạt SLA.

## 4. Phân Hệ Cầu Nối: Tích Hợp Python RL và Java Simulator bằng Py4J

Để giải quyết khác biệt ngôn ngữ, kiến trúc sử dụng **Py4J Gateway**. Phân hệ này được thiết kế để hỗ trợ quá trình **Hyperparameter Tuning** (tinh chỉnh siêu tham số) của Giai đoạn 2 bằng cách cung cấp đường truyền dữ liệu ổn định ở tốc độ cực cao, cho phép chạy song song hàng nghìn Episodes.

- **Bên Java (Backend):** Khởi chạy một GatewayServer. CloudSim Plus sẽ tạm dừng mô phỏng (pause simulation) mỗi khi có một tác vụ mới đi vào hàng đợi và yêu cầu quyết định lập lịch.
- **Bên Python (RL Agent):** Kết nối đến Gateway, truy xuất danh sách các Host. Tại tầng Python này, dự án sẽ tích hợp sẵn các framework theo dõi (như Weights & Biases hoặc MLflow) để ghi nhận điểm số phần thưởng qua từng bước, chuẩn bị cho quá trình điều chỉnh learning\_rate, discount\_factor ở Giai đoạn 2.

*Cấu trúc kết nối Python cơ bản:*

Python

```
from py4j.java_gateway import JavaGateway
import wandb # Chuẩn bị cho tuning Giai đoạn 2

# Kết nối với CloudSim Plus đang chạy ở background
gateway = JavaGateway()
cloudsim_broker = gateway.entry_point.getBroker()
datacenter = gateway.entry_point.getDatacenter()

# Lấy trạng thái hệ thống
hosts = datacenter.getHostList()
#... Xử lý vector trạng thái cho AI...
```

## 5. Phân Hệ Trí Tuệ Nhân Tạo: Học Tăng Cường Đa Mục Tiêu (MORL)

Khi giao tiếp đã được thiết lập, quá trình tối ưu hóa mâu thuẫn (Năng lượng vs. SLA) được mô hình hóa thành một Môi trường tùy chỉnh thông qua thư viện **MO-Gymnasium** (Python). Môi trường này được thiết kế để tương thích với các thuật toán DRL tiên tiến như Proximal Policy Optimization (PPO), vốn đã được chứng minh hiệu quả trong việc cân bằng động giữa thời gian hoàn thành (makespan) và năng lượng.

### 5.1. Mô Hình Hóa Bài Toán (MOMDP Formulation)

**Không gian Trạng thái (State Space):**

Được biểu diễn dưới dạng một ma trận trích xuất từ CloudSim qua Py4J:

- Mức độ sử dụng CPU/RAM/GPU hiện tại của từng Host.
- Năng lượng đang tiêu thụ hiện tại của từng Host ( $P_{active}$  và  $P_{idle}$ ).
- Chi tiết tác vụ (Cloudlet) đang chờ được phân bổ.

**Không gian Hành động (Action Space):**

Hành động  $a_t \in \{1, 2, \dots, N\}$  với  $N$  là số lượng Host. Quyết định đặt Vm/Cloudlet vào Host số  $j$ .

### 5.2. Hàm Mục Tiêu Kép (Reward Vector)

Tại mỗi bước  $t$ , môi trường MO-Gymnasium Python sẽ gửi Action sang CloudSim. CloudSim

chạy tiếp mô phỏng, tính toán độ chênh lệch, và trả về vector phần thưởng  $\mathbf{R}_t =^T$  :

**Mục tiêu 1: Tối thiểu hóa năng lượng tăng thêm ( $R_{energy}$ )**

$$R_{energy} = -(E_{t+1} - E_t)$$

Tác nhân sẽ học được rằng việc bật một Host mới đang ở chế độ ngủ (Sleep) sẽ tốn điện năng nền, do đó sẽ ưu tiên dồn tác vụ vào các Host đang hoạt động.

**Mục tiêu 2: Hình Phạt Vi Phạm SLA ( $R_{SLA}$ )**

SLA được định nghĩa chặt chẽ để hệ thống RL có thể tối thiểu hóa thời gian hoàn thành (makespan) cũng như các vi phạm liên quan đến thời hạn (deadline) của tác vụ.

$$R_{SLA} = -\lambda \max(0, CompletionTime_{predicted} - Deadline)$$

Hệ số phạt  $\lambda$  được điều chỉnh theo tham số qos của tác vụ (ví dụ: tác vụ có độ trễ nhạy cảm sẽ bị phạt rất nặng nếu dự đoán trễ hạn).

## 6. Kế Hoạch Triển Khai Định Hướng Nền Tảng (Giai Đoạn 1)

Lộ trình triển khai Giai đoạn 1 (từ tuần 1 đến tuần 15) được thiết kế tập trung vào việc tạo ra môi trường thử nghiệm vững chắc, sẵn sàng bàn giao cho các công đoạn phát triển thuật toán:

- Tuần 1 - 4 (Thiết lập Nền tảng CloudSim Plus):**
  - Cài đặt môi trường Java (JDK 17+) và Maven.
  - Khởi tạo dự án CloudSim Plus, thiết lập kịch bản Datacenter với PowerModel cơ bản.
- Tuần 5 - 8 (Tích hợp Dữ liệu & Xây dựng Gateway):**
  - Viết module Java đọc và parse dữ liệu Alibaba Trace. Thiết kế bộ lọc tạo các kịch bản tải (Low, High, Burst).
  - Cài đặt thư viện Py4J. Thiết kế các API để Python đọc trạng thái Host và gửi lệnh gán Cloudlet.
- Tuần 9 - 13 (Mô hình hóa AI & Xây dựng Baseline):**
  - Xây dựng file cấu hình môi trường chuẩn gym.Env bằng MO-Gymnasium trong Python.
  - Lập trình lớp VmAllocationPolicyK8sDefault trong Java để đóng vai trò Kube-scheduler mặc định.
- Tuần 14 - 15 (Thiết lập Baseline & Chuẩn hóa Môi trường):**
  - Chạy mô phỏng toàn bộ dữ liệu mẫu thông qua bộ lập lịch Baseline (Default K8s Scheduler) để thu thập mốc năng lượng và SLA ban đầu.

- Cấu hình các công cụ theo dõi (Wandb/Tensorboard) trong Python.
- Viết báo cáo Giai đoạn 1: Chứng minh môi trường mô phỏng hoạt động trơn tru, API giao tiếp giữa Python và Java có độ trễ thấp, và dữ liệu Baseline đã sẵn sàng để các tác nhân DRL bắt đầu học hỏi ở Giai đoạn 2.